CUESTIONARIO 11

Lambdas y streams

Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What is the output?

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
public class StreamTest {
    public static void main(String[] args) {
        List<String> items = Arrays.asList("apple", "banana", "cherry", "date");
        List<String> result = items.stream()
                                   .filter(s -> s.startsWith("a"))
                                   .collect(Collectors.toList());
        System.out.println(result);
    }
}
```

Seleccione una:

- a. [apple, banana]
- b. [banana, cherry]
- c. [apple] ✓
- d. [apple, cherry]

Justificación: filter conserva solo los elementos para los que el predicado es true, y únicamente "apple" empieza con "a". "banana" y "date" contienen la letra a, pero startsWith solo mira el inicio de la cadena. Salida verificada con JDK 21: [apple].

Fuente: Java.pdf #101

La respuesta correcta es: [apple]

Mi justificación
El filtro deja solo las frutas con a así que solo quedamos con apple

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Ordenamiento {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Kevin");
        names.add("Andrea");
        names.add("Laila");
        names.add("Josias");

        List<String> namesOrder = names.stream()
                                       .sorted()
                                       .collect(Collectors.toList());

        System.out.println(namesOrder);
    }
}
```

Seleccione una:

- a. Compilation error.
- b. Andrea, Josias, Kevin, Laila ✓
- c. Laila, Kevin, Josias, Andrea
- d. Kevin, Andrea, Laila, Josias

Justificación: sorted() sin argumentos usa el orden natural de String (alfabético), así que la lista resultante es [Andrea, Josias, Kevin, Laila]. c es el orden inverso y d el orden de inserción. Se asume que el fragmento incluye los imports de java.util y java.util.stream.

Mi justificación
El método sorted() ordena la lista names en orden alfabético así que queda Andrea, Josias, Kevin, Laila.

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Ordenamiento {
    public static void main(String[] args) {
        List<Integer> numbersList = new ArrayList<>();
        numbersList.add(3);
        numbersList.add(1);
        numbersList.add(9);
        numbersList.add(8);

        List<Integer> numbersOrder = numbersList.stream()
                                                .sorted(Collections.reverseOrder())
                                                .collect(Collectors.toList());

        System.out.println(numbersOrder);
    }
}
```

Seleccione una:

- a. 9, 8, 3, 1 ✓
- b. 3, 1, 9, 8
- c. Compilation error
- d. 1,3,8,9

Justificación: sorted(Collections.reverseOrder()) ordena de forma descendente, así que se obtiene [9, 8, 3, 1]. La opción d es el orden natural sin comparador y la b el orden de inserción. Se asume que el fragmento incluye los imports de java.util y java.util.stream.

Fuente: Quiz Desarrollo APX 2 #8

La respuesta correcta es: 9, 8, 3, 1

Mi justificación
El método reverseOrder() ordena numbersList en orden inverso así que obtenemos 9,8,3,1

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál es el resultado del siguiente código?

```
public class App {
    public static void main(String[] args) {
        String[] fruits = {
            "banana",
            "apple",
            "pears",
            "grapes"
        };
        Arrays.sort(fruits, (a, b) -> a.compareTo(b));
        for (String s : fruits) {
            System.out.print(" " + s);
        }
    }
}
```

Seleccione una:

- a. banana apple pears grapes
- b. apple banana grapes pears ✓
- c. pears grapes apple banana
- d. Lanza una excepción en tiempo de ejecución

Justificación: El comparador (a, b) -> a.compareTo(b) es el orden natural (lexicográfico) de String, así que el arreglo queda apple, banana, grapes, pears y se imprime con un espacio delante de cada palabra. No se lanza ninguna excepción porque ningún elemento es null. La opción a es el orden original, que solo saldría si no se ordenara.

Fuente: Quiz Desarrollo APX 1 #1

La respuesta correcta es: apple banana grapes pears

Mi justificación
El método sort(a,b) con a.compareTo(b) ordena la lista en orden alfabético así que queda apple banana grapes pears.

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Cuál es el resultado al ejecutar el siguiente codigo?

```
import java.util.Arrays;
import java.util.List;
public class Test {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        long count = numbers.stream()
                .map(n -> n * 2)
                .filter(n -> n > 5)
                .count();
        System.out.println(count);
    }
}
```

Seleccione una:

- a. 0
- b. 2
- c. 3 ✓
- d. 4
- e. Compilation fails.

Justificación: map duplica cada número (2, 4, 6, 8, 10) y filter conserva los mayores que 5 (6, 8, 10), así que count() devuelve 3. Quien responde 2 olvida que el 6 también cumple la condición, y quien responde 0 aplica el filtro sobre los valores originales en lugar de los transformados.

Fuente: Simulación APX V4 #3 · también aparece en: Java.pdf #12

La respuesta correcta es: 3

Mi justificación
En orden. La lista original es (1,2,3,4,5) .map(n -> n*2) multiplica cada número por 2 así que tenemos (2,4,6,8,10). .filter(n -> n>5) deja solo aquellos que son mayores a 5 así que deja (6,8,10). .count() cuenta los elementos así que imprime 3.

Pregunta 6

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

Dado el siguiente código: ¿Qué imprime en pantalla el código?

```
import java.util.List;
import java.util.function.Predicate;

class Person {
    String name;
    public Person(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return name;
    }
}

public class Test {
    static List<Person> filter(List<Person> input, Predicate<Person> predicate) {
        return input.stream().filter(predicate).toList();
    }
    public static void main(String[] args) {
        var a = new Person("John");
        var b = new Person("Jane");
        var c = new Person("Mark");
        var input = List.of(a, b, c);
        var output = filter(input, p -> !p.name.startsWith("J"));
        System.out.println(output);
    }
}
```

Seleccione una:

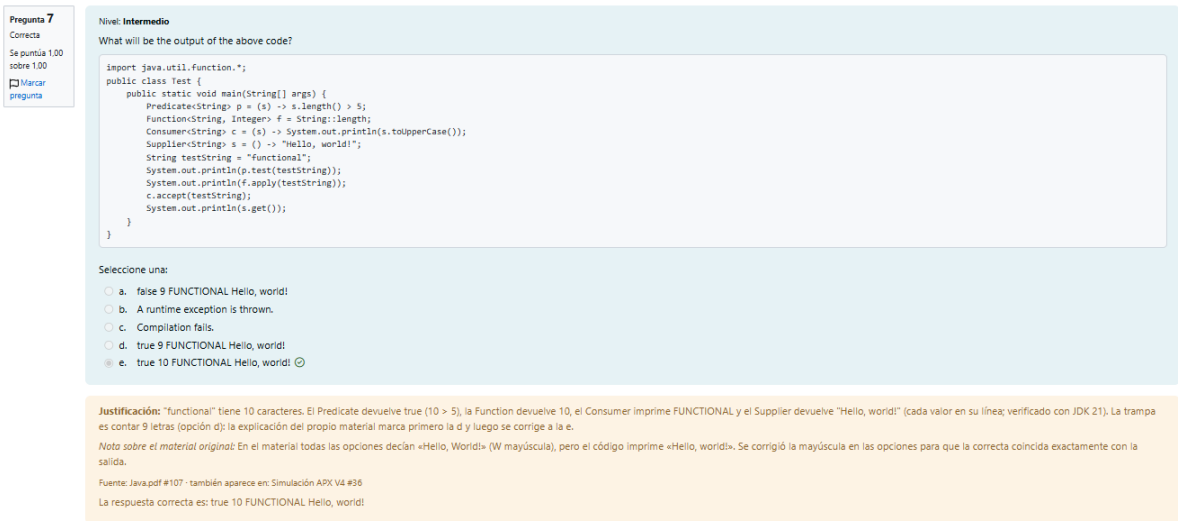
- a. [John, Jane, Mark]
- b. null
- c. [Jane]
- d. []
- e. [Mark] ✓
- f. [John, Jane]
- g. [John]

Justificación: El predicado conserva a las personas cuyo nombre NO empieza con "J", así que se descartan John y Jane y queda [Mark]; toString() devuelve el nombre. Stream.toList() existe desde Java 16 y var desde Java 10. La opción [John, Jane] es la trampa para quien pasa por alto la negación !. Salida verificada con JDK 21: [Mark].

Fuente: Simulación APX V1 #38 · también aparece en: Simulación APX V4 #8, Java.pdf #20

La respuesta correcta es: [Mark]

Mi justificación
El filtro deja de la lista solo aquellos que no inician con J (por el !), así que deja solo a Mark.



Pregunta 7
Correcta
Se puntúa 1.00 sobre 1.00
Marcar pregunta

Nivel: **Intermedio**

What will be the output of the above code?

```
import java.util.function.*;
public class Test {
    public static void main(String[] args) {
        Predicate<String> p = (s) -> s.length() > 5;
        Function<String, Integer> f = String::length;
        Consumer<String> c = (s) -> System.out.println(s.toUpperCase());
        Supplier<String> s = () -> "Hello, world!";
        String testString = "functional";
        System.out.println(p.test(testString));
        System.out.println(f.apply(testString));
        c.accept(testString);
        System.out.println(s.get());
    }
}
```

Seleccione una:

a. false 9 FUNCTIONAL Hello, world!
b. A runtime exception is thrown.
c. Compilation fails.
d. true 9 FUNCTIONAL Hello, world!
e. true 10 FUNCTIONAL Hello, world!

Justificación: "functional" tiene 10 caracteres. El Predicate devuelve true (10 > 5), la Function devuelve 10, el Consumer imprime FUNCTIONAL y el Supplier devuelve "Hello, world!" (cada valor en su línea; verificado con JDK 21). La trampa es contar 9 letras (opción d): la explicación del propio material marca primero la d y luego se corrige a la e.

Nota sobre el material original: En el material todas las opciones decían «Hello, World!» (W mayúscula), pero el código imprime «Hello, world!». Se corrigió la mayúscula en las opciones para que la correcta coincida exactamente con la salida.

Fuente: Java.pdf #107 · también aparece en: Simulación APX V4 #36

La respuesta correcta es: true 10 FUNCTIONAL Hello, world!

Mi justificación
En orden. “functional” tiene 10 letras así que pasa el test del Predicate que da true cuando el string tiene longitud mayor a 5. Imprime true. Ya que su longitud es de 10, la Function evaluada en el string imprime 10. El Consumer cambia todo el string a mayúsculas así que imprime FUNCTIONAL. El Supplier imprime “Hello World”. La opción e) imprime todo esto.

Pregunta 8

Incorrecta

Se puntúa 0,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuál será la salida del código anterior?

```
import java.util.function.Predicate;
class Foo {
    int i;
    Foo(int i) {
        this.i = i;
    }
}
public class Test {
    static boolean compare(Foo foo, Predicate<Foo> predicate) {
        return predicate.test(foo);
    }
    public static void main(String[] args) {
        Predicate<Foo> p = f -> f.i > 0; // line 1
        p = p.or(f -> f.i > 0).and(f -> f.i < 0); // line 2
        boolean b = compare(new Foo(1), p); // line 3
        System.out.println(b);
    }
}
```

Seleccione una:

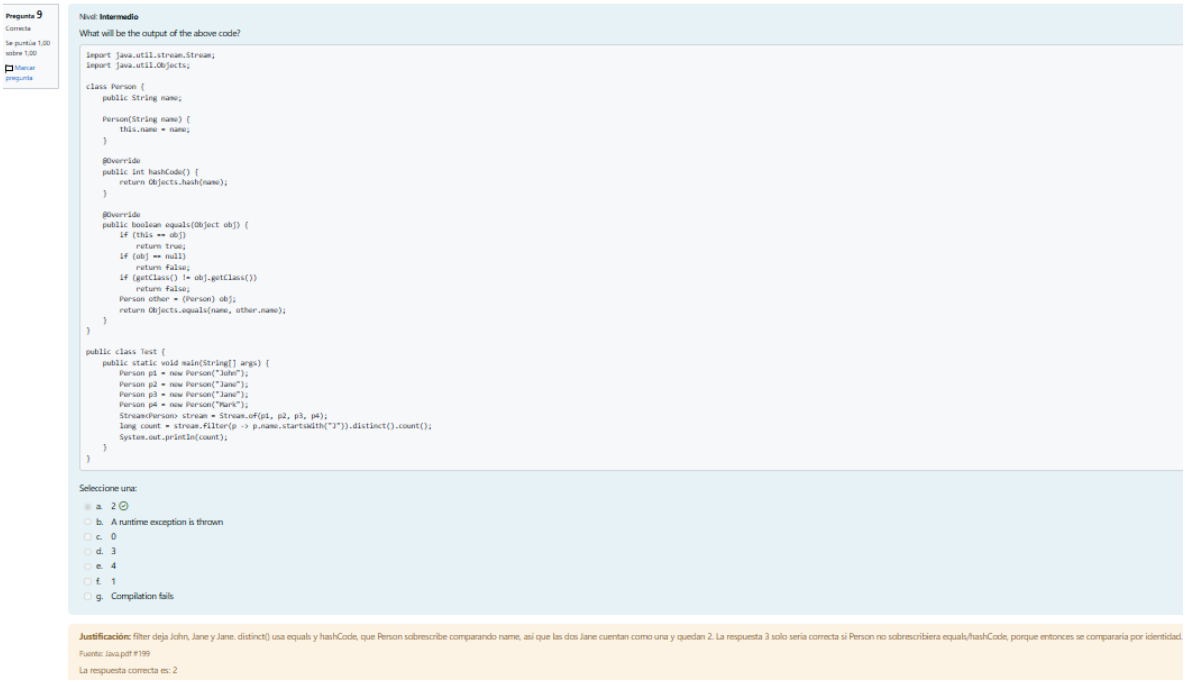
a. Compilación falla en la línea 1.
b. Compilación falla en la línea 3.
c. true
d. false
e. Compilación falla en la línea 2. ⊗

Justificación: p.or(f -> f.i > 0) sigue equivaliendo a 'i > 0', y .and(f -> f.i < 0) exige además 'i < 0'. Para Foo(1) queda true && false = false. Las tres líneas marcadas compilan: las lambdas se infieren como Predicate<Foo> y or/and son métodos default de Predicate.

Fuente: Cuestionario APX V3 #5 · también aparece en: Java.pdf #138

La respuesta correcta es: false

Mi justificación
La verdad no le entendí a este cuando lo respondí por primera vez.



Pregunta 9
Correcte
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Intermedio**

What will be the output of the above code?

```
import java.util.stream.Stream;
import java.util.Objects;

class Person {
    public String name;

    Person(String name) {
        this.name = name;
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Person other = (Person) obj;
        return Objects.equals(name, other.name);
    }
}

public class Test {
    public static void main(String[] args) {
        Person p1 = new Person("John");
        Person p2 = new Person("Jane");
        Person p3 = new Person("Jane");
        Person p4 = new Person("Mark");
        Stream<Person> stream = Stream.of(p1, p2, p3, p4);
        long count = stream.filter(p -> p.name.startsWith("J")).distinct().count();
        System.out.println(count);
    }
}
```

Seleccione una:

- a. 2 ✓
- b. A runtime exception is thrown
- c. 0
- d. 3
- e. 4
- f. 1
- g. Compilation fails

Justificación: filter deja John, Jane y Jane. distinct() usa equals y hashCode, que Person sobrescribe comparando name, así que las dos Jane cuentan como una y quedan 2. La respuesta 3 solo sería correcta si Person no sobrescribiera equals/hashCode, porque entonces se compararía por identidad.

Fuente: Java.pdf #199

La respuesta correcta es: 2

Mi justificación
El filtro deja solo a aquellos que inician con J, pero distinct() deja solo a los únicos así que elimina a la segunda Jane y deja solo a John y Jane, y el count() cuenta e imprime estos 2 elementos.



Pregunta **10**

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Avanzado**

Given: What will be the output of the above code?

```
interface IFun {
int calculate(int i1, int i2);
}
public class Test {
public static void main(String[] args) {
IFun f = (i1, i2) -> i1 + i2; // line 1
int i1=1, i2 = 2; // line 2
IFun g = (i1, i2) -> i1 * i2; // line 3
System.out.println(g.calculate(i1, i2)); // line 4
}
}
```

Seleccione una:

- a. Compilation fails on line 4.
- b. Compilation fails on line 2.
- c. 2
- d. Compilation fails on line 1.
- e. Compilation fails on line 3. ✓
- f. 3

Justificación: Los parámetros de una lambda no pueden llamarse igual que variables locales que siguen en su ámbito. En line 3, i1 e i2 ya existen (se declararon en line 2), así que la compilación falla ahí: javac da «variable i1 is already defined in method main(String[])» y lo mismo para i2, 2 errores y ambos en line 3 (verificado con JDK 21). En line 1 sí es legal, porque en ese punto i1 e i2 todavía no están declaradas.

Nota sobre el material original: En el material, line 4 decía «g.calculate(i1, ii)», una errata de i2. Con «ii», javac también da error en line 4 y la opción a sería igualmente cierta. Se corrigió a i2 para que la única respuesta correcta sea la e, que es la clave del material.

Fuente: Java.pdf #99 · también aparece en: Simulación APX V4 #33

La respuesta correcta es: Compilation fails on line 3.

Mi justificación
Las variables i1 e i2 ya existen en la línea 2, a partir de ahí no pueden usarse como parámetros de lambda, así que la línea 3 así que no compile.

RESULTADO = 9/10 = 9.0